

Série 1 : Applications de l'IA

Licence d'excellence Intelligence Artificielle

Pr A. Belangour

Exercice 1 : Classification des données

Pour chaque élément ci-dessous, indiquez s'il s'agit de données **structurées**, **semi-structurées** ou **non structurées**. Justifiez votre réponse en 1 ou 2 phrases.

1. Un fichier CSV contenant les relevés de température quotidiens dans plusieurs villes.
2. Un enregistrement audio d'une réunion d'équipe.
3. Un fichier XML contenant des informations sur des commandes clients.
4. Une base de données PostgreSQL d'une entreprise de e-commerce.
5. Une collection de photographies satellite non annotées.
6. Un flux de logs serveur au format JSON.
7. Un document PDF scanné et non indexé.

Exercice 2 : Analyse critique d'un Dataset

On vous confie deux jeux de données pour entraîner des modèles d'IA.

Dataset A : Un ensemble de 10 000 avis clients sur des restaurants, chacun accompagné d'une note sur 5 étoiles. Les avis sont en texte libre.

Dataset B : Un ensemble de 500 images de panneaux de signalisation, toutes prises de jour par beau temps, dans une seule ville européenne.

Pour chaque dataset :

1. Quel est le type de données principal ?
2. Quelle tâche d'IA pourrait-on envisager avec ces données ?
3. Identifiez au moins une limitation ou un biais potentiel dans ce dataset.
4. Proposez une amélioration concrète pour le rendre plus robuste.

Exercice 3 : Prétraitement conceptuel

On considère le dataset brut suivant, extrait d'une plateforme de location de vélos :

I D	Ville	Température (°C)	Humidité (%)	Nb_vélos_disponibles	Jour_semaine	Météo
1	Paris	25	60	45	Lundi	Ensoleillé
2	Lyon	18	85	-5	Mercredi	Pluvieux
3	Marseille	30		120	Samedi	Ensoleillé
4	Paris	22	55	45	Mardi	Nuageux
5	Bordeaux	12	90	8	Dimanche	Pluvieux
6	Lille	8	95	0	Lundi	Neigeux
7	Nantes		70	32	Jeudi	

1. **Nettoyage** : Identifiez toutes les anomalies dans ce tableau (valeurs aberrantes, manquantes, incohérentes). Proposez un traitement pour chacune.
2. **Encodage** : Proposez une méthode pour transformer les colonnes "Ville", "Jour_semaine" et "Météo" en variables numériques. Justifiez vos choix.
3. **Normalisation** : Appliquez la formule de standardisation (z-score) à la colonne "Température". Montrez les calculs pour les deux premières lignes.
4. **Création de feature** : Proposez une nouvelle colonne pertinente à créer à partir des données existantes, qui pourrait aider à prédire le nombre de vélos disponibles.

Exercice 4 : Éthique et conformité

Pour chaque situation ci-dessous, identifiez le problème éthique ou légal et proposez une solution.

1. Une entreprise française développe un assistant vocal entraîné uniquement sur des voix masculines.
2. Une startup collecte des données de navigation d'enfants sans le consentement des parents.

3. Un hôpital souhaite partager des dossiers médicaux anonymisés avec un laboratoire de recherche, mais l'anonymisation est mal faite et permet de ré-identifier certains patients.
4. Un système de notation de crédit utilise un dataset où les demandes des quartiers défavorisés sont sous-représentées.

TP 1 : Collecte et prétraitement de données textuelles par web scraping

Objectif : Mettre en œuvre une chaîne complète de collecte, nettoyage et prétraitement de données textuelles à partir du web.

Contexte

Vous travaillez pour une startup qui souhaite développer un outil d'analyse des offres d'emploi au Maroc. L'objectif est d'identifier les compétences les plus demandées, les fourchettes de salaires, et les tendances du marché. Vous devez constituer un dataset textuel à partir d'offres d'emploi collectées sur le web.

Partie 1 : Mise en place de l'environnement

1. Installez les bibliothèques Python suivantes : requests, beautifulsoup4, selenium, pandas, numpy, re, nltk OU spacy.
2. Créez une structure de dossiers pour organiser votre projet :

text

```
projet_emploi/  
├─ scraping/  
│   ├─ scrape_emploi.py  
│   └─ config.py  
├─ data/  
│   ├─ brutes/  
│   ├─ nettoyes/  
│   └─ analyses/  
├─ rapports/  
└─ requirements.txt
```

Partie 2 : Analyse du site cible et stratégie de scraping

On considère un site d'offres d'emploi (vous pouvez choisir un site réel comme Rekrute, [Emploi.ma](https://emploi.ma), ou utiliser un site factice pour l'exercice).

1. **Analyse préalable** : Visitez le site et identifiez :
 - La structure des URLs pour les pages de recherche.
 - La structure HTML d'une page de liste d'offres.
 - La structure HTML d'une page de détail d'une offre.
2. **Respect des règles** : Consultez le fichier `robots.txt` du site. Notez les règles à respecter.
3. **Planification** : Listez les informations que vous souhaitez extraire pour chaque offre (titre, entreprise, lieu, date de publication, description, compétences requises, salaire si disponible).

Partie 3 : Scraping des URLs des offres

1. **Script de base** : Écrivez un premier script qui récupère la page de recherche et extrait les URLs de toutes les offres d'emploi présentes.
2. **Pagination** : Ajoutez la gestion de la pagination pour collecter les URLs des offres sur plusieurs pages.
3. **Politesse** : Implémentez un délai aléatoire entre les requêtes pour ne pas surcharger le serveur.
4. **Stockage intermédiaire** : Sauvegardez la liste des URLs collectées dans un fichier CSV pour éviter de devoir les récupérer à nouveau.

Partie 4 : Scraping des détails de chaque offre

1. **Extraction des détails** : Pour chaque URL collectée, écrivez une fonction qui :
 - Télécharge la page de détail.
 - Extrait les informations définies (titre, entreprise, description, etc.).
 - Gère les cas où certaines informations sont manquantes.
2. **Gestion des erreurs** : Implémentez un système de gestion des erreurs (timeout, page non trouvée, structure HTML différente) avec des tentatives de reprise.
3. **Progression** : Ajoutez une barre de progression (avec `tqdm`) pour visualiser l'avancement du scraping.
4. **Sauvegarde** : Stockez toutes les offres collectées dans un fichier JSON structuré (une entrée par offre) dans le dossier `data/brutes/`.

Partie 5 : Nettoyage des données textuelles

Le dataset brut contient du texte brut avec beaucoup de bruit (balises HTML résiduelles, caractères spéciaux, espaces multiples, etc.).

1. **Nettoyage basique** : Écrivez une fonction de nettoyage qui :
 - Supprime les balises HTML résiduelles.
 - Supprime les caractères spéciaux non désirés (gardez la ponctuation de base).
 - Normalise les espaces (supprime les espaces multiples, les retours à la ligne excessifs).
 - Corrige les problèmes d'encodage (accents, caractères spéciaux).
2. **Tokenisation** : Utilisez NLTK ou spaCy pour tokeniser le texte des descriptions.
3. **Suppression des stop words** : Supprimez les mots vides (stop words) en français.
4. **Lemmatisation** : Appliquez une lemmatisation pour réduire les mots à leur forme canonique.
5. **Sauvegarde** : Stockez les descriptions nettoyées et tokenisées dans un nouveau fichier CSV structuré dans le dossier `data/nettoyees/`.

Partie 6 : Analyse exploratoire et enrichissement

1. **Extraction de compétences** : Créez une fonction qui identifie les compétences techniques mentionnées dans chaque description (utilisez une liste de mots-clés ou une approche plus avancée).
2. **Analyse des fréquences** : Générez un nuage de mots (word cloud) des termes les plus fréquents dans les offres.
3. **Statistiques** : Produisez un petit rapport avec :
 - Nombre total d'offres collectées.
 - Répartition géographique (par ville).
 - Top 10 des compétences demandées.
 - Analyse des salaires (si disponibles).
4. **Visualisation** : Créez des graphiques avec matplotlib ou seaborn pour illustrer ces statistiques.

Partie 7 : Éthique et documentation

1. **Respect des conditions** : Rédigez un paragraphe expliquant comment vous avez respecté les conditions d'utilisation du site et la législation sur la protection des données (loi 09-08 au Maroc).
2. **Limites du dataset** : Identifiez les biais potentiels de votre collecte (ex: surreprésentation de certaines villes, de certains secteurs, période de collecte limitée).
3. **Reproductibilité** : Documentez précisément les étapes de votre scraping pour que quelqu'un d'autre puisse reproduire votre travail.

Livrables attendus

- Tous les scripts Python commentés.
- Un extrait du fichier JSON brut (10 premières offres).
- Le fichier CSV final avec les données nettoyées.
- Un rapport au format PDF (3-4 pages) présentant :
 - La méthodologie de scraping.
 - Les difficultés rencontrées et leurs solutions.
 - Les résultats de l'analyse exploratoire (graphiques inclus).
 - Une réflexion éthique sur le scraping réalisé.

TP 2 : Constitution d'un dataset d'images pour la classification d'oiseaux

Objectif : Mettre en œuvre une chaîne complète de collecte, nettoyage et prétraitement d'images à partir du web.

Contexte

Vous êtes data scientist dans une jeune entreprise qui souhaite développer une application mobile permettant d'identifier les espèces d'oiseaux à partir de photos. Vous devez constituer un dataset d'images de qualité pour entraîner le modèle de classification.

Partie 1 : Mise en place de l'environnement

1. Installez les bibliothèques Python suivantes
: requests, BeautifulSoup4, selenium, pillow, opencv-python, imagehash, pandas, numpy, matplotlib, tqdm.
2. Créez une structure de dossiers pour organiser votre projet :

```
projet_oiseaux/  
├─ scraping/  
│   ├─ scraper_oiseaux.py  
│   └─ config.py  
├─ data/  
│   ├─ brutes/  
│   ├─ nettoyes/  
│   └─ finales/  
├─ scripts/  
│   ├─ nettoyage.py  
│   ├─ pretraitement.py  
│   └─ augmentation.py  
└─ rapports/
```

Partie 2 : Collecte d'images par web scraping

Vous allez collecter des images pour 5 espèces d'oiseaux : "Hibou grand-duc", "Flamant rose", "Martin-pêcheur", "Cygne tuberculé", "Pic vert".

1. **Script de scraping** : Écrivez un script Python qui utilise BeautifulSoup ou Selenium pour rechercher et télécharger des images depuis un moteur de recherche (ex: Bing Images, Flickr). Pour chaque espèce, téléchargez au moins 100 images.
2. **Gestion des requêtes** : Implémentez un délai aléatoire entre les requêtes pour ne pas surcharger le serveur.
3. **Stockage** : Organisez les images téléchargées dans des sous-dossiers correspondant à chaque espèce (dans `data/brutes/`).
4. **Journalisation** : Créez un fichier log qui enregistre les URLs téléchargées et les échecs.
5. **Robustesse** : Gérez les timeouts, les erreurs HTTP et les images qui ne se téléchargent pas correctement.

Partie 3 : Nettoyage du dataset

Le dataset brut contient probablement des images inutilisables (doublons, images trop petites, images corrompues, mauvaises espèces, images avec filigranes).

1. **Suppression des doublons** : Écrivez une fonction qui utilise `imagehash` pour détecter et supprimer les images en double dans chaque dossier.
2. **Filtrage par taille** : Supprimez toutes les images dont la largeur ou la hauteur est inférieure à 200 pixels.
3. **Détection d'images corrompues** : Utilisez Pillow ou OpenCV pour vérifier que chaque image peut être ouverte correctement. Supprimez celles qui sont corrompues.
4. **Détection d'images avec filigrane (bonus)** : Proposez une méthode simple pour détecter les images contenant des filigranes ou du texte superposé (par exemple, en analysant la netteté des contours ou la présence de zones à faible variance).
5. **Vérification manuelle assistée** : Générez un petit échantillon aléatoire d'images (10 par classe) et affichez-les avec matplotlib. Notez si certaines sont mal classées et déplacez-les ou supprimez-les manuellement.

Partie 4 : Prétraitement des images

1. **Redimensionnement** : Redimensionnez toutes les images à une taille uniforme de 224x224 pixels (taille standard pour beaucoup de modèles de deep learning).
2. **Normalisation** : Convertissez les valeurs des pixels de l'intervalle [0, 255] à [0, 1] (ou [-1, 1] selon votre préférence).
3. **Format uniforme** : Convertissez toutes les images en format JPG avec une compression standard.
4. **Organisation finale** : Enregistrez les images prétraitées dans `data/finales/` en conservant la structure par dossiers d'espèces.

Partie 5 : Data Augmentation

Pour améliorer la robustesse du futur modèle, vous allez augmenter artificiellement la taille du dataset.

1. **Script d'augmentation** : Créez un script qui génère des variations de vos images :
 - Rotation aléatoire (entre -15 et +15 degrés)
 - Flip horizontal aléatoire
 - Zoom aléatoire (facteur entre 0.9 et 1.1)
 - Légère variation de luminosité et de contraste
2. **Génération** : Pour chaque image originale, générez 5 images augmentées.
3. **Organisation** : Stockez les images augmentées dans un dossier séparé ou ajoutez un suffixe au nom de fichier pour les identifier.
4. **Visualisation** : Affichez une grille d'images originales avec leurs versions augmentées pour vérifier le résultat et la pertinence des transformations.

Partie 6 : Division du dataset

Pour préparer l'entraînement d'un modèle, vous devez diviser votre dataset en ensembles d'entraînement, de validation et de test.

1. **Répartition** : Créez un script qui répartit les images de chaque espèce selon les proportions suivantes :
 - 70% pour l'entraînement
 - 15% pour la validation
 - 15% pour le test

2. **Structure** : Organisez ces ensembles dans une structure de dossiers claire :
text

```
dataset_final/  
├─ train/  
│   ├─ hibou/  
│   ├─ flamant/  
│   └─ ...  
├─ val/  
│   ├─ hibou/  
│   └─ ...  
└─ test/  
    ├─ hibou/  
    └─ ...
```

Partie 7 : Analyse et documentation

1. **Statistiques** : Produisez un rapport détaillé avec :
 - Nombre d'images collectées par espèce (avant nettoyage)
 - Nombre et pourcentage d'images supprimées lors du nettoyage (avec ventilation par motif : doublons, taille, corruption)
 - Nombre d'images après augmentation
 - Répartition finale dans train/val/test
2. **Visualisations** : Créez des graphiques pour illustrer ces statistiques (diagrammes en barres, camemberts).
3. **Biais potentiels** : Rédigez une analyse critique de votre dataset :
 - Les images proviennent-elles principalement d'une région géographique ?
 - Y a-t-il des variations suffisantes (saisons, angles, arrière-plans, éclairages) ?
 - Certaines espèces sont-elles plus faciles à trouver que d'autres ?
 - Y a-t-il un risque de biais de confirmation (images d'oiseaux toujours dans des positions typiques) ?
4. **Respect des droits** : Documentez les sources des images et discutez des aspects légaux (copyright, conditions d'utilisation des sites, droit à l'image).

Partie 8 : Script d'exploration rapide (bonus)

Créez un petit script qui permet d'explorer visuellement le dataset :

- Afficher une grille aléatoire d'images d'une espèce donnée.
- Afficher la distribution des tailles d'images originales (avant redimensionnement).
- Identifier les espèces avec le moins d'images pour prioriser une collecte supplémentaire.

Livrables attendus

- Tous les scripts Python commentés (scraping, nettoyage, prétraitement, augmentation, division).
- Un fichier README expliquant comment exécuter les scripts dans le bon ordre.
- Le dataset final organisé (ou un lien de téléchargement s'il est volumineux).
- Un rapport au format PDF (4-5 pages) présentant :
 - La méthodologie complète (de la collecte à la préparation finale).
 - Les difficultés rencontrées et les solutions apportées.
 - Les statistiques détaillées (avec graphiques).
 - Une analyse critique des limites et biais du dataset.
 - Une réflexion éthique sur la collecte d'images et leur utilisation.